

Spark RDDs: Paper Review

Rohan Gore - rmg9725

Collaborators: ChatGPT 5

References

1. “Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing” (original paper)

1 Problem Statements Tackled

1. Existing systems exchange intermediate results through stable storage, incurring excessive disk I/O, serialization, and replication costs.
2. Iterative algorithms (PageRank, K-Means, logistic regression) and interactive queries require efficient in-memory reuse of partial results.
3. Distributed shared memory and key-value stores allow fine-grained updates but impose heavy fault-tolerance overhead via replication or logging.
4. A unified abstraction was needed that supported coarse-grained operations, fast recovery, deterministic recomputation, and explicit control over persistence.

2 Key Design Principles

1. **Immutable, Deterministic Data Structures:** RDDs represent datasets as immutable, partitioned collections where each transformation (e.g., `map`, `filter`, `join`) produces a new RDD rather than mutating existing data. Because transformations are deterministic and side-effect-free, the scheduler can reason about RDD behavior precisely, enabling safe recomputation, parallelization, and consistent semantics even across distributed executors.
2. **Lineage-Based Fault Tolerance:** Instead of replicating partitions across nodes, each RDD maintains a lineage graph—a record of the sequence of coarse-grained transformations used to construct it. Upon partition loss, Spark reconstructs only the missing partitions by replaying the lineage on the minimal set of ancestor RDDs. This approach drastically reduces fault-tolerance overhead compared to fine-grained logging or multi-replica storage, and fits naturally with immutable data.
3. **Lazy Evaluation Through DAG Construction:** Transformations do not execute immediately; they add nodes and edges to a logical DAG that captures data dependencies. When an action is invoked (e.g., `collect`, `count`, `saveAsTextFile`), Spark analyzes the DAG, identifies shuffle boundaries, and compiles it into stages of pipelined operations. This allows global optimization—such as collapsing multiple narrow transformations—and avoids unnecessary materialization.
4. **Explicit Persistence Control:** Users may choose to persist intermediate RDDs in memory, in memory with serialization, on disk, or replicate them across nodes. These options let applications trade off memory footprint, CPU overhead, and recovery speed. Persisting RDDs is especially valuable for iterative workloads (e.g., machine learning, graph algorithms) where repeated re-scanning of input data would otherwise dominate execution time.
5. **User-Controlled Partitioning:** RDDs expose partitioners (e.g., `HashPartitioner`, `RangePartitioner`) that determine how data is distributed across the cluster. Co-partitioned datasets enable efficient joins and reduce communication during shuffles. Developers can explicitly control partition counts and partitioning strategies to improve locality, load balance, and performance predictability for iterative or shuffle-heavy workflows.

3 Architecture and Components

1. **Representation of RDDs:** Every RDD implements five core components: (1) a list of partitions describing logical data fragments, (2) dependencies describing the relationship between parent and child RDDs, (3) a `compute(partition)` function that materializes the contents of a partition on demand, (4) an optional partitioner for key/value RDDs, and (5) preferred locations that guide scheduling. Spark distinguishes between *narrow dependencies*—where each child partition depends on a small number of parent partitions—and *wide dependencies*, where many-to-many relationships require data shuffling.
2. **Stage Construction and Job Scheduling:** Spark converts the DAG of RDD transformations into a physical execution plan by grouping sequences of narrow dependencies into stages. Each stage executes as a pipeline on a set of partitions without requiring materialization of intermediate outputs. Wide dependencies introduce shuffle stages, where intermediate records are hash-partitioned, written to local disk, and served to downstream tasks. Scheduling uses *delay scheduling* to improve locality, allowing tasks to wait briefly for preferred nodes. Shuffle outputs are materialized and tracked in the shuffle manager, simplifying recovery because reduce tasks can re-fetch only the required map outputs instead of rebuilding entire RDDs.
3. **Interactive Execution via the Scala Interpreter:** Spark’s tight integration with the Scala REPL enables interactive data analysis on clusters. When a user defines a transformation in the REPL, Spark compiles the closure into a JVM class, distributes this class to all executors, and serializes referenced variables using a modified closure-capture mechanism. This dynamic code shipping supports iterative experimentation and rapid prototyping on datasets that may span terabytes.
4. **Memory Management and Persistence:** RDDs may be cached using one of several storage levels: deserialized Java objects (fastest for computation), serialized byte arrays (memory-efficient), or disk-backed storage when memory is scarce. Spark employs an LRU eviction mechanism at the RDD level, ensuring that evicted partitions belong to the least recently used RDD while avoiding eviction of data from the same RDD mid-scan, which protects against thrashing. Memory management is designed to preserve locality by retaining reused RDDs in memory across stages.

4 Related Work

1. **Dataflow Systems (MapReduce, Dryad, Ciel)** These systems express parallel computations but exchange intermediate datasets only through stable storage, limiting performance for iterative and interactive workloads.
2. **Language-Integrated Parallel Collections (DryadLINQ, FlumeJava)** These systems provide functional APIs but treat datasets as files or ephemeral query results. They cannot persist intermediate data across computations. Spark generalizes this model by backing the API with RDDs.
3. **Iterative Computation Frameworks (Pregel, HaLoop, Twister)** These systems support iterative workloads but embed data reuse implicitly and restrict computation patterns. RDDs provide an explicit, general-purpose abstraction that supports both iterative and interactive analytics.
4. **Shared Mutable-State Systems (Piccolo, DSM, RAMCloud)** These systems enable fine-grained updates but rely on costly checkpointing or replication. RDDs avoid these overheads by using immutable, coarse-grained transformations and lineage for recovery.
5. **Lineage and Provenance Systems** Prior work captures lineage for debugging or reproducibility, but Spark extends lineage across computations to enable low-cost recovery of distributed in-memory datasets.

5 Conclusion

RDDs provide a powerful, fault-tolerant abstraction for distributed in-memory computation. Their immutable, coarse-grained, lineage-based design overcomes the limitations of disk-bound dataflow systems and fine-grained shared memory models. By supporting fast recovery, optimized data placement, explicit persistence, and a rich set of transformations, RDDs unify interactive analytics, iterative algorithms, and general dataflow workloads within a single programming model. Spark operationalizes these ideas to deliver a scalable, efficient, and expressive cluster computing engine.